CS & IT ENGINEERING

Compiler Design

Lexical Analysis

Principles, Techniques

Lecture No.- 02

By- Venkat sir

Recap of Previous Lecture

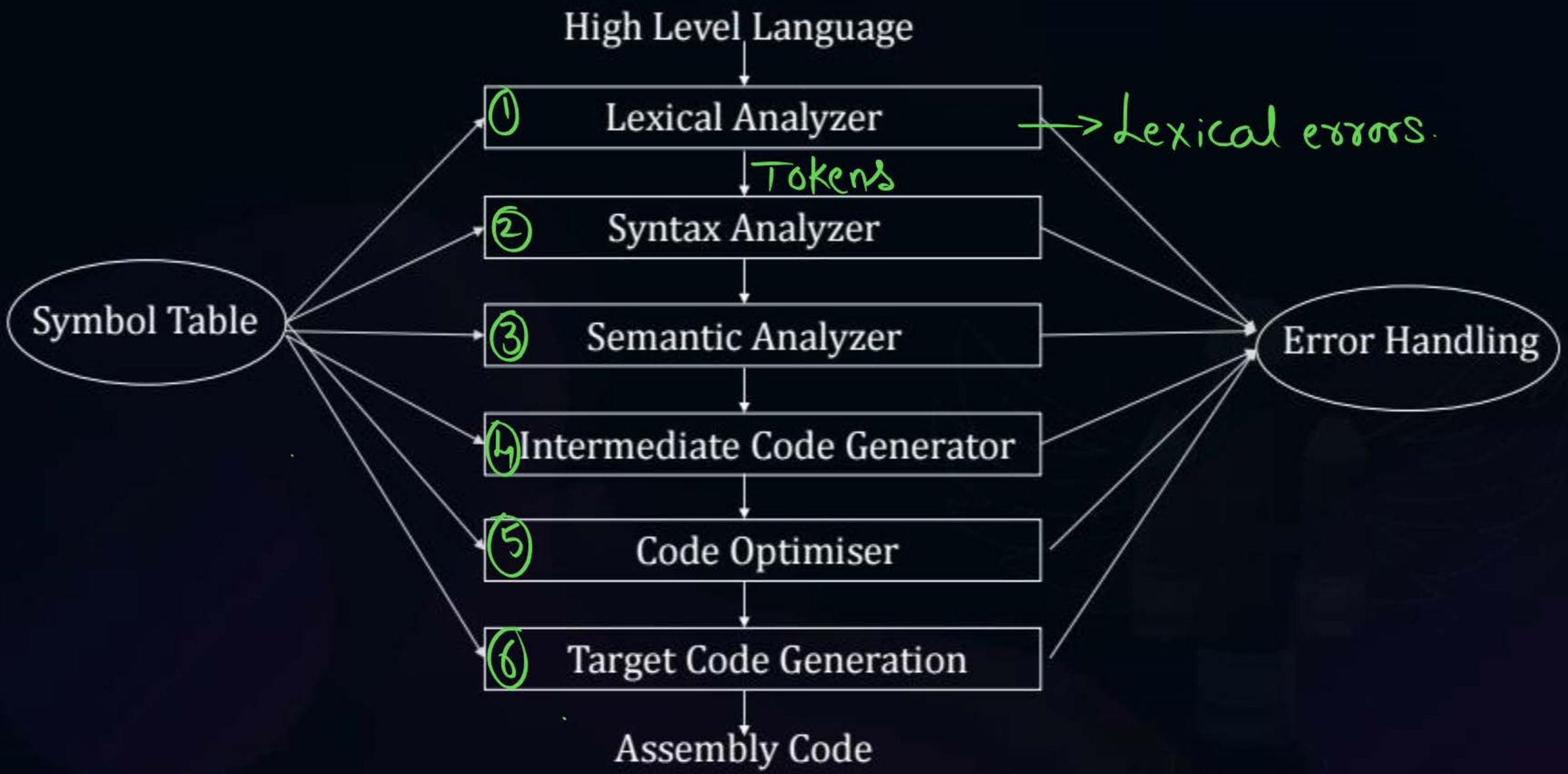
- Topic: Introduction of Compiler Design
- Topic: Intermediate Code Generation phase
- Topic: The role of lexical analyzer
- Topic: Syntax Analysis Introduction
- Topic: LR Parsing

Topics to be Covered

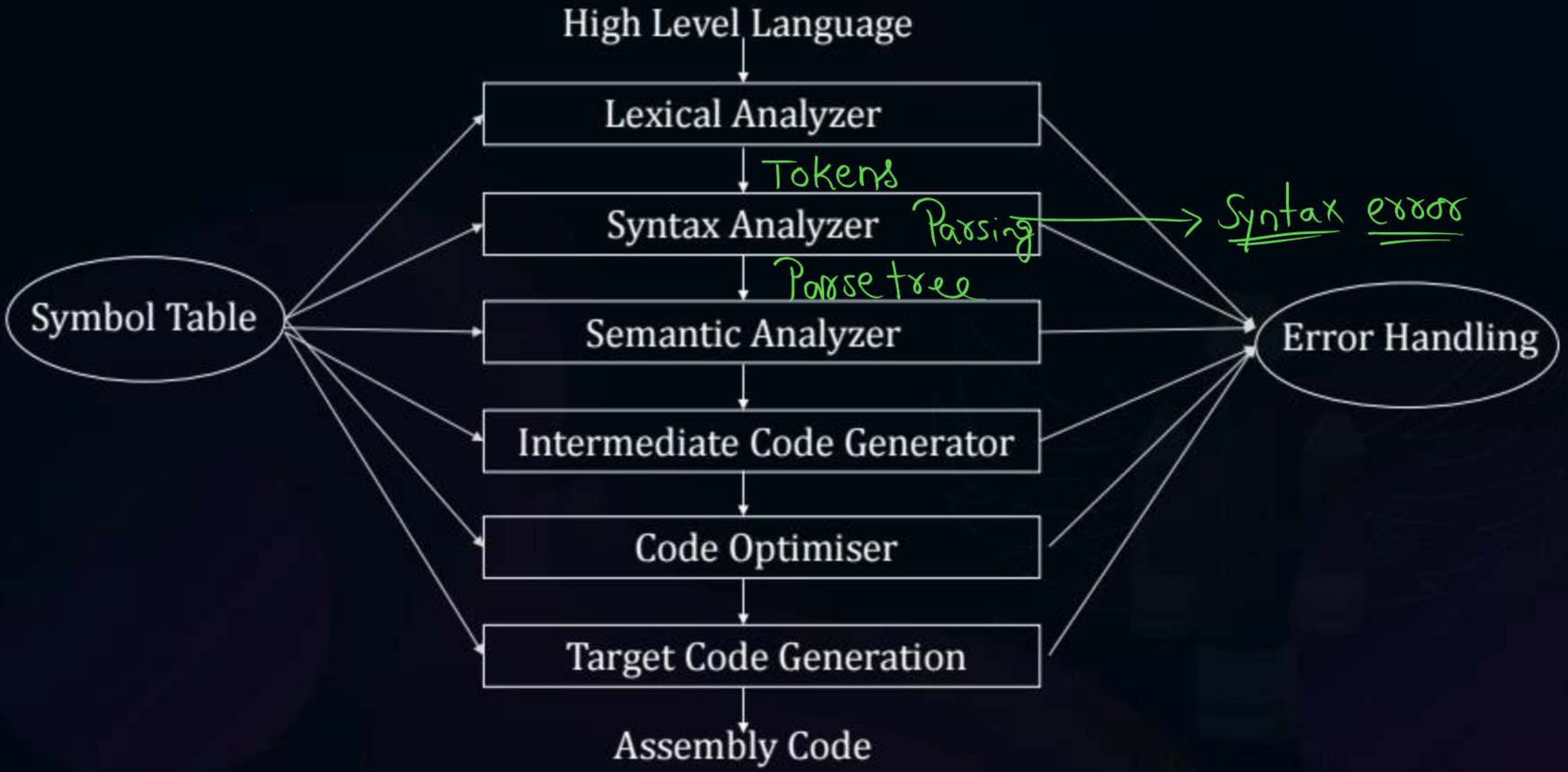
✓ Phases of Compilers.

- Topic ① Lexical Analysis
- Topic Parsing
- Topic Syntax Directed Translation
- Topic Intermediate code generation
- Topic Code Optimization (Dataflow Analysis)
- Topic Runtime Environment

Topic : Introduction of Compiler Design



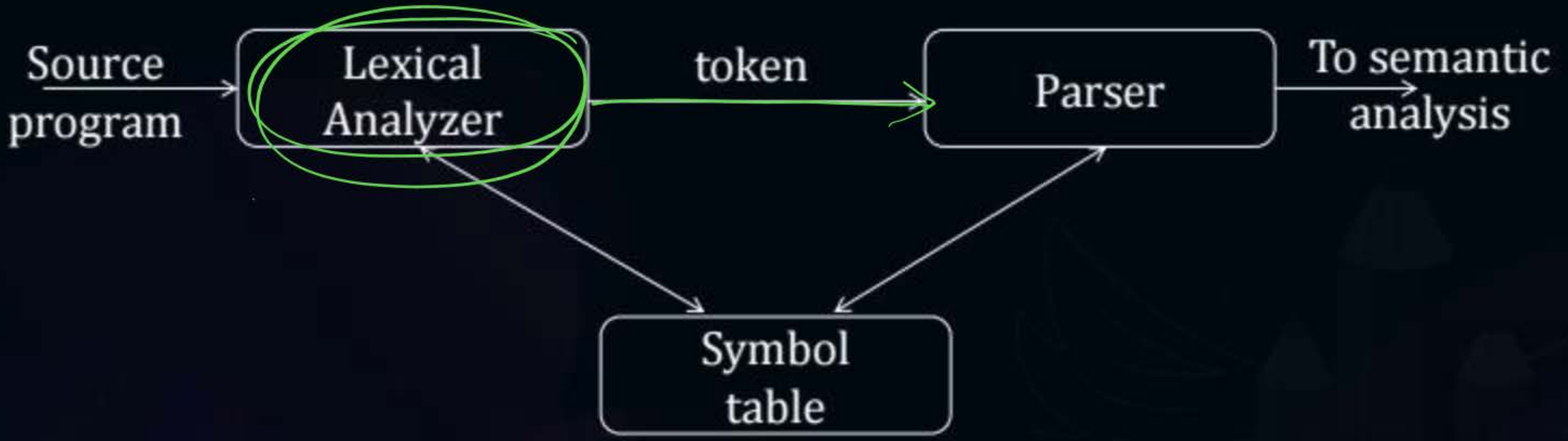
Topic : Lexical Analysis



Topic : Functionality of Lexical Analysis

1. It reads source program as stream of characters
2. It produces tokens as output
3. It detects lexical errors.
4. It eliminates comment lines
5. It eliminates white space characters
6. It constructs symbol table

Topic : The Role of Lexical Analyzer



Topic : The Role of Lexical Analyzer

Why to separate Lexical analysis and parsing

- Simplicity of design
- Improving compiler efficiency
- Enhancing compiler portability

Topic : The Role of Lexical Analyzer

- **Token:** Token is a sequence of characters that can be treated as a single logical entity.
- Typical tokens are,
- 1. Identifiers
- 2. keywords
- 3. operators
- 4. special symbols
- 5. constants

Topic : Compiler Design

tokens

1. Keywords (int, float, for)
2. Identifiers (xyz, ab)
3. Constants(10, 2,0 30)
4. Operators(+, *)
5. Special symbols ((,.) {};)

++

+

{Lexical error}

(1) int 1xyz ; (K.W.)

(2) char b[] = "Compiler"

Lexical error

(3) char a = 'b

Lexical error

{
}
);

① int (id) abc (id) ; (S.S) } 3 tokens

② a | = | b | ; | * | c | ; } 7 tokens

③ int a, a, a ; } no Lexical error

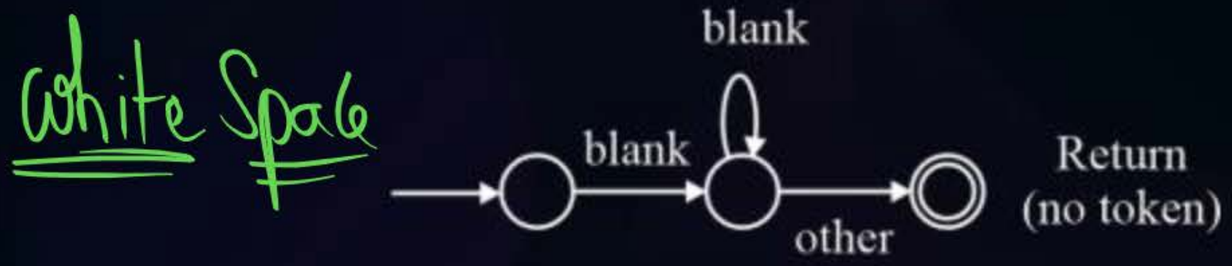
④ int a = "Compiler" ; } no lexical error

Topic : Comment

Comment

1. x = x + z; /* comment */ } 6 tokens
2. x = /* comment */ / y + z; } 6
3. x = y + z;

int □ abc ;



[NAT]

#Q. How many tokens are generated by the lexical analyzer, if the following program has not lexical error?

```
main()
{
        int x, y;
        fl/*gate*/oat z;
        x = /*exam*/10;
        y = 20;
}
```

22 tokens

no token

no token

```
main()
{
        int x, y;
        fl oat z;
        x = 10;
        y = 20;
}
```

[NAT]

#Q. Consider the C program

```
main()
{
    int x = 10;
    x = x+y+z; /* compiler Design*/
}
```

How many tokens are identified by lexical analyzer?

18 tokens

[NAT]

#Q. Consider the following program segment:

```
main ()
{
        int a, b;
            a= 5 + 8 +;
            printf("%d", a);
        }
```

24 tokens

The number of token present in the above program segment

[MCQ]

#Q. Which of the following is not a token in C language?

A. Semicolon ✓

B. Identifier ✓

C. Keyword ✓

D. White space

(D)

(Q) Which of the following C Program having Lexical error?

(a)
```
main()
{
  int a,b;
  c=a+b;
}
```
no lexical error

(b)
```
main()
{
  int a,a,a,a;
}
```
no L.E

(c)
```
main()
{
  int a,b;
  char c="Hello";
  a=b+c;
}
```
int + string

no Lexical error

(d) none

① Tokens Generation

② Lexical error detection

[NAT]

#Q. Consider the following program segment:

```
main()
{
    int a, b, c;
    a = 50;
    b = &a;
    printf("%d", b);
}
```

The number of tokens in the above C code are _____.

28 tokens

Topic : Syntax

Analysis } Parsing Parser

int a,b,c,d ;

Syntax

- Structure of Program statements known as syntax.

Parsing

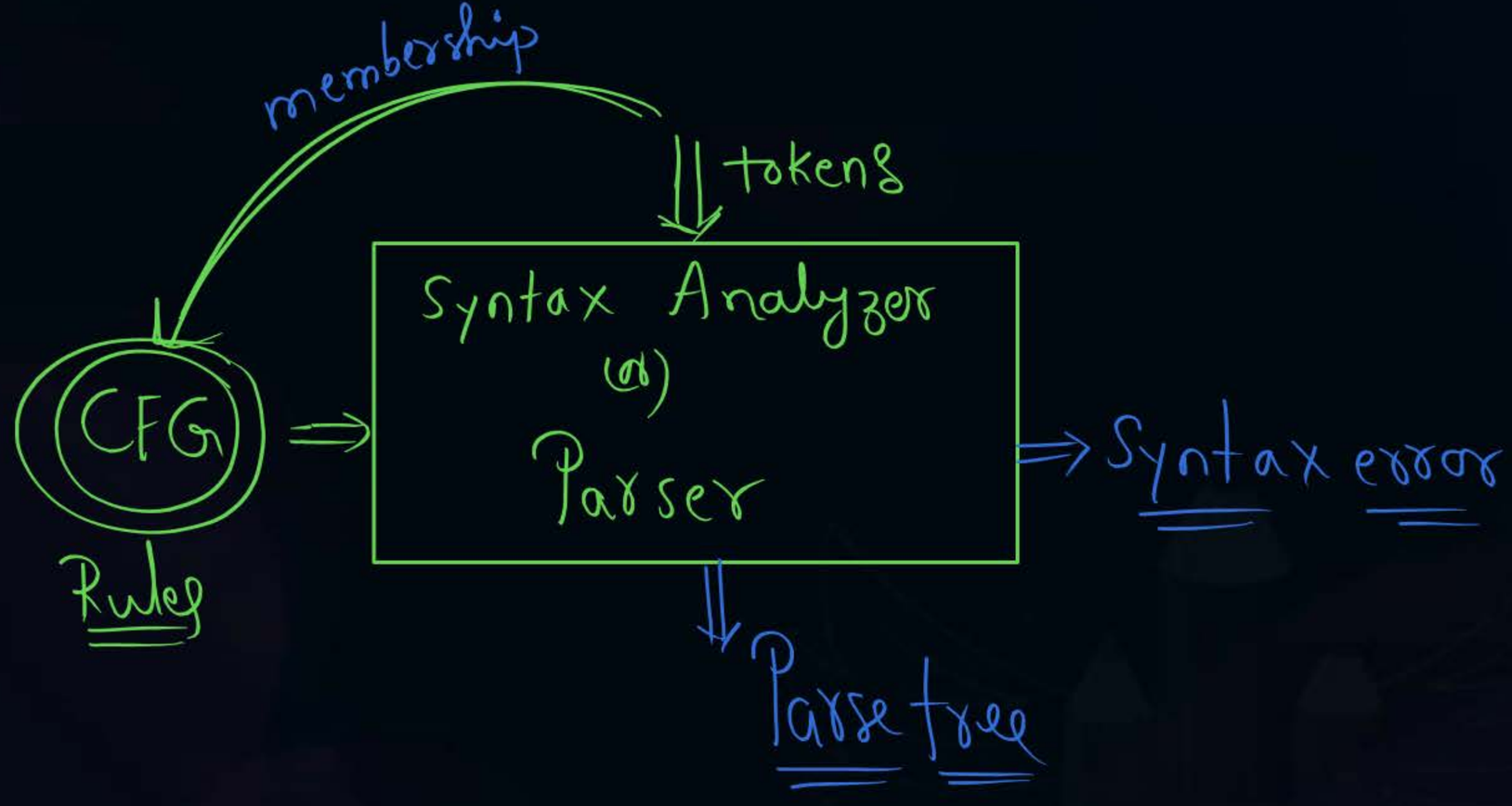
ij w ∈ Grammar ?

- Checking given string is member of given grammar (or) not knowns as parsing.

Parser (or) syntax Analyzer

Verify

- It is a program that takes tokens and CFG as input and tokens are member or not.
- If tokens are member parser produces parse tree as output. Otherwise syntax error is generated.



1. CFG
2. Parsing Algorithms

CFG $[A \to \alpha]$

$\alpha \in (V+T)^*$

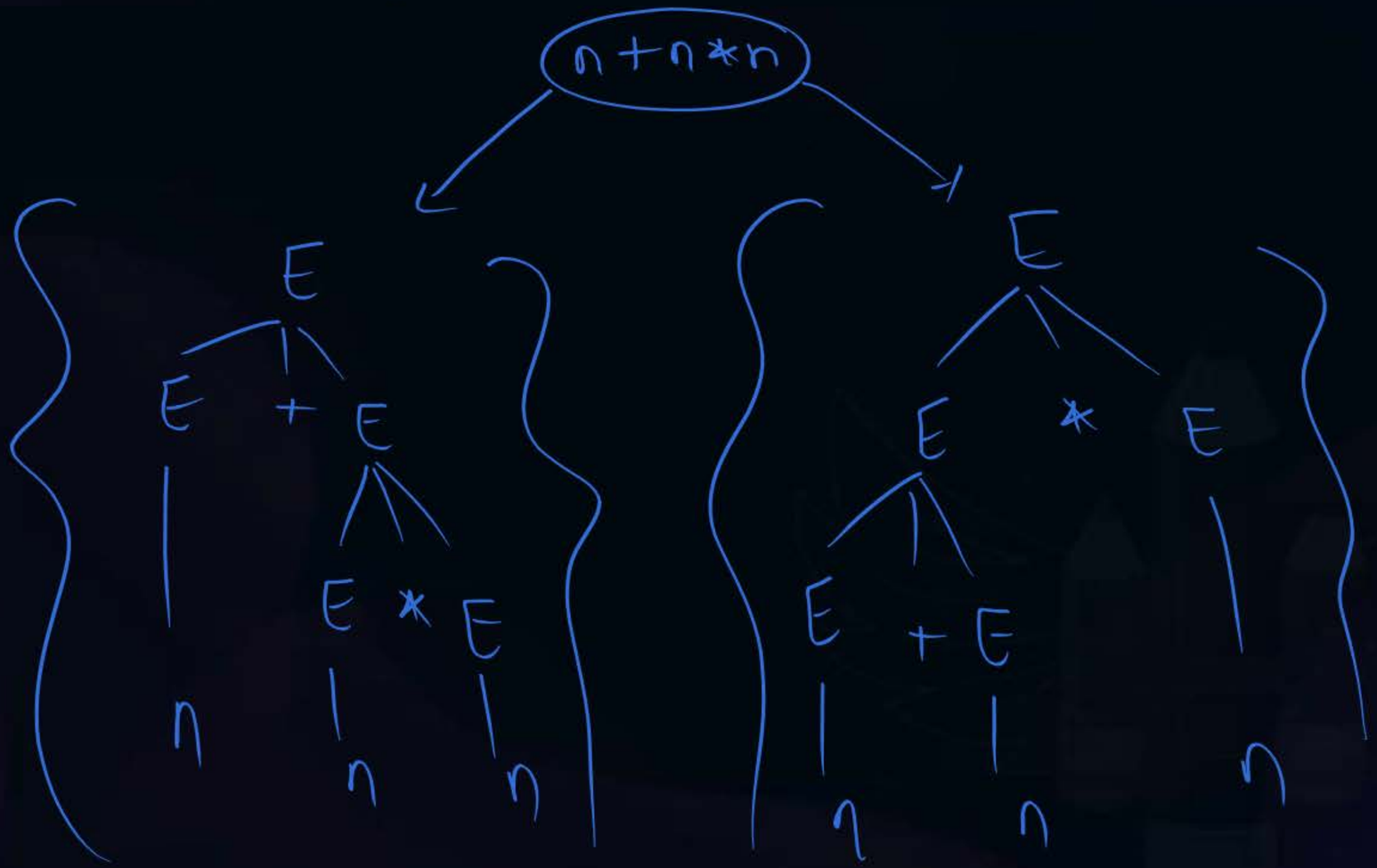
① Ambiguous Grammars are not good for Parser design.

Ex:- {CFG}

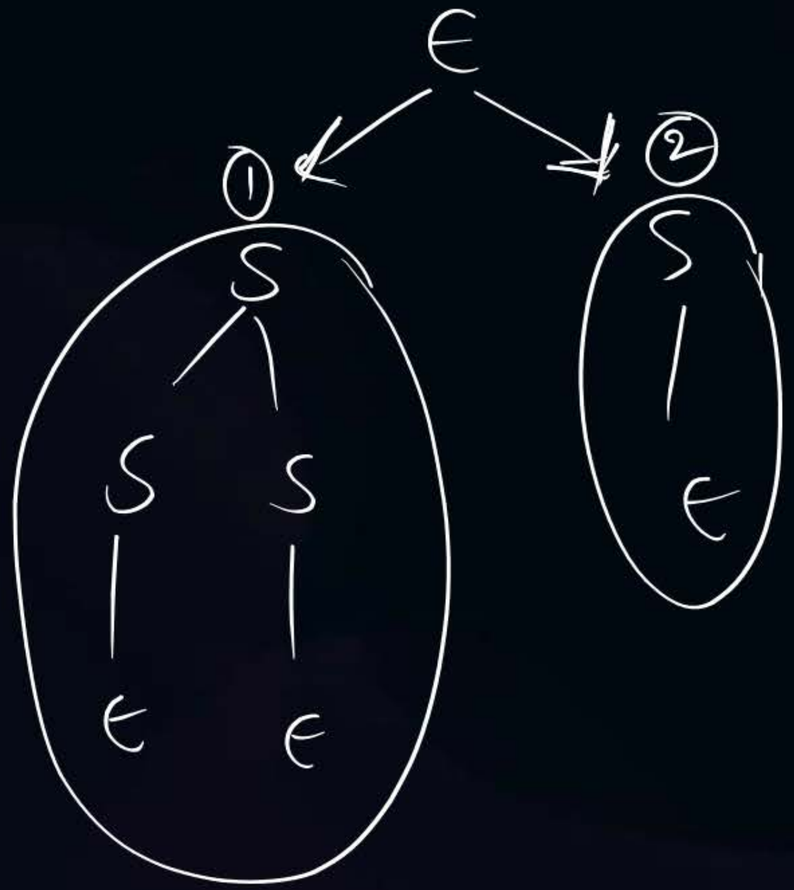
$$\begin{cases} S \to AB \\ A \to a \\ B \to b \end{cases}$$

Ex :-

$E \rightarrow E+E \mid E*E \mid n$ } NO



Ex: $S \rightarrow SS \mid \epsilon$ } Not good for Parser



Ex

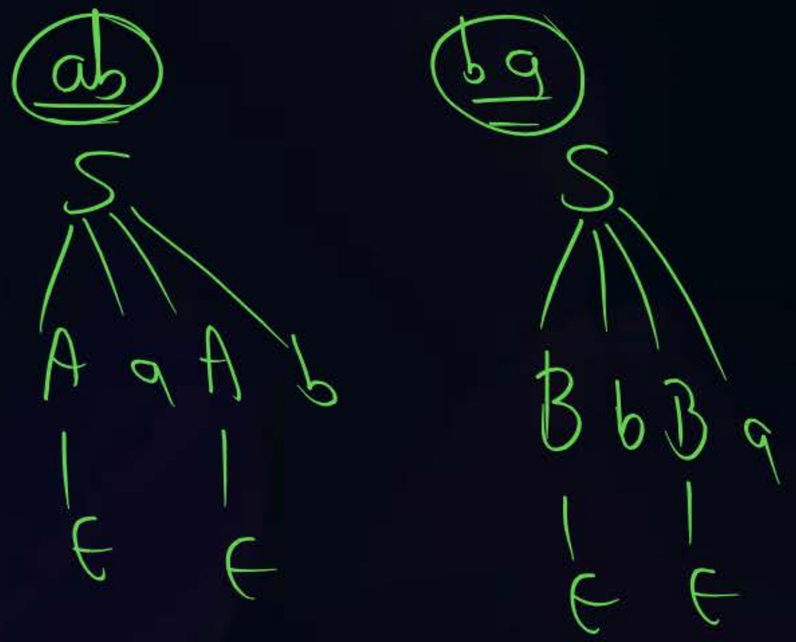
$S \to AaAb \mid BbBa$

$A \to \epsilon$

$B \to \epsilon$

Unambiguous Grammar

Suitable for Parser ✓

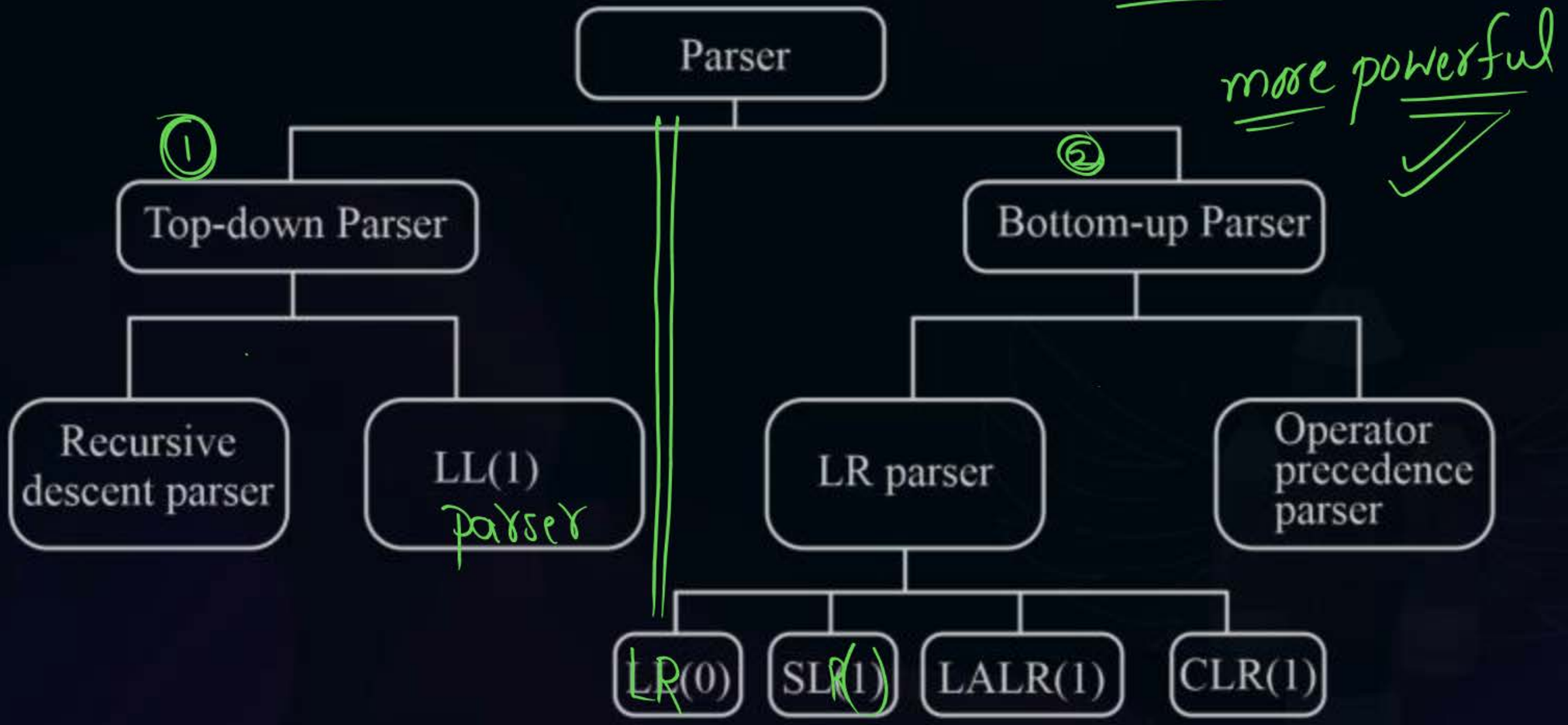


Topic : Parsing ALGORITHMS

- Parser (or) syntax analyzer is a deterministic pushdown automata.
- To design parser two types of algorithms exist known as
  1. Bottom up parsing algorithm.
  2. Top down parsing algorithm.

Topic : Parsing Algorithms

Pushdown Automata (DPDA)



more powerful

2 mins Summary

- **Topic** One
- **Topic** Two
- **Topic** Three
- **Topic** Four
- **Topic** Five

THANK - YOU